

**Merge Conflict
Programmer's Manual
CS450 – Spring 2025**

Table of Contents

C Files	1
comexec.c	1
comexec()	1
shutdown()	1
help()	2
version()	2
get_time()	3
set_time()	3
get_date()	4
set_date()	4
include/mpx/comexec.h	5
include/string.h	6
include/mpx/serial.h	7
include/mpx/device.h	8
include/sys_req.h	9
include/mpx/io.h	10
include/mpx/interrupts.h	10
include/stdlib.h	11
include/ctype.h	12
include/memory.h	12
 comhand.c	 14
comhand()	14
include/mpx/comexec.h	14
include/mpx/device.h	15
include/mpx/serial.h	16
include/sys_req.h	17
 ctype.c	 19
isdigit()	19

include/ctype.h	19
serial.c	20
buffer_refresh()	20
include/serial.h	20
include/mpx/io.h	21
include/string.h	22
include/sys_req.h	23
stdlib.c	25
itoa()	25
bcdToChar()	25
intToBCD()	26
include/stdlib.h	26
include/ctype.h	27
string.c	28
contains()	28
substr()	28
isNumeric()	29
charCount()	29
indexOf()	29

C Files

comexec.c

- Comexec.c is where the execution of all user entered commands occurs. It holds functionality for version, help, get_time, get_date, set_time, set_date, and shutdown.

comexec()

- int comexec(char buf[])
 - Authors: Tanner Forbes, Chris Jones, Izaak Whetsell, Evan Humphrey
 - This function executes the different system commands after reading the buffer in from comhand()
 - Parameters:
 - char buf[] is an array of characters that the user has typed
 - Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

shutdown()

- int shutdown(void)
 - Authors: Izaak Whetsell, Chris Jones
 - This function terminates the command handling sequence
 - Parameters:
 - None

- Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

help()

- int help(void)
- Author: Chris Jones
- This function prints a list of each available command
- Parameters:
 - None
- Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

version()

- int version(void)
- Authors: Chris Jones, Evan Humphrey
- This function prints the current version of the system
- Parameters:
 - None
- Return Values:

- This function returns an integer that represents the status of the functions completion.
- 1 is to shutdown the operating system
- 0 is to continue running

`get_time()`

- `int get_time(void)`
- Author: Tanner Forbes
- This function utilizes the real time clock registers to display the current time
- Parameters:
 - None
- Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

`set_time()`

- `int set_time(char buf[])`
- Author: Chris Jones
- This function utilizes the real time clock registers to set the current time in the system
- Parameters:
 - `char buf[]` is an array of characters that the user has typed. This array should contain the proper time format (HH:MM:SS)

- Valid Parameters:
 - Valid hour values include: 01-24
 - Valid minute values include: 00-59
 - Valid second values include: 00-59
- All valid entries for single digits MUST include a beginning 0 before the parameter
- Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

get_date()

- int get_date(void)
- Author: Tanner Forbes
- This function utilizes the real time clock registers to retrieve and display the current date
- Parameters:
 - None
- Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

set_date()

- int set_date(char buf[])
- Author: Tanner Forbes

- This function utilizes the real time clock registers to set the current date in the system
- Parameters:
 - char buf[] is an array of characters that the user has typed. This array should contain the proper date format (MM/DD/YYYY) and should accurately match a calendar.
 - Example requirements include no months greater than 12, no days greater than 31, each month has designated 28-30-31 day scheme, leap year, and year is within the 21st Century.
- Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

include/mpx/comexec.h

- This file contains all functions that execute system commands when called in the terminal

Function Prototypes:

int comexec(char buf[])

- This function is called to execute any command called in the system, it will scan through the input and determine whether a command is called correctly or incorrectly

int shutdown(void)

- This function will terminate the command handling sequence in the system and shut down any processes executing

int help(void)

- This function will print a list of each available command from the current point in the system

int version(void)

- This function prints the compilation date and version of the system

int get_time(void)

- This function retrieves the date from the real time clock (RTC) and displays the current date to the user

int set_time(char buf[])

- This function sets the date of the system using the real time clock (RTC) and displays the results of the command

int get_date(void)

- This function gets the current time of the system using the real time clock (RTC) and displays the time to the user

int set_date(char buf[])

- This function sets the date of the system using the real time clock (RTC) and displays the results of the command

include/string.h

- This file contains all functions pertaining to string manipulation and identification

`int contains(const char *str1, const char *str2)`

- This function indicates whether str1 contains str2

`char* substr(char* str1, int index)`

- This function will return the substring of a passed-in string beginning at a starting index

`int isNumeric(char* str)`

- This function determines whether a string is entirely numeric

`int charCount(char* str, char c)`

- This function finds the number of occurrences of a specific character in a given string

`int indexOf(char* str, char searched_char)`

- This function finds the specific index of a desired character in a string

`include/mpx/serial.h`

This header file contains the functions and constants used in `serial_poll()`

Constants include:

- `UP_ARROW`

- DOWN_ARROW
- LEFT_ARROW
- RIGHT_ARROW
- DELETE_KEY
- BACKSPACE
- NEW_LINE
- CARRIAGE_RETURN
- ESCAPE_SEQUENCE

All of which represent the character sequence of each of their respective keys.

Function Prototypes:

`int serial_init(device dev)`

- This function initializes the devices connected to the OS for input and output

`int serial_out(device dev, const char *buffer, size_t len)`

- This function writes the contents of the buffer to the serial port

`int serial_poll(device dev, char *buffer, size_t len)`

- This function reads the contents of a string to the serial port

`void buffer_refresh(char *buffer, int buf_length, int pos)`

- This function refreshes the buffer in the terminal window after a key is pressed

`include/mpx/device.h`

This header file contains the constants used to identify the device I/O ports in the system

Enums:

device

COM1,

COM2,

COM3,

COM4

Each of these constants contain a memory address which gives the location of the I/O ports to be written to/read from

include/sys_req.h

This file contains all system request functions and constants

Enums:

op_code

EXIT,

IDLE,

READ,

WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- **INVALID_OPERATION**
- **INVALID_BUFFER**
- **INVALID_COUNT**

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:

`int sys_req(op_code op, ...)`

- This function requests a kernel operation within the system to perform an action based on the input parameters.

`include/mpx/io.h`

This header file contains kernel macros to write and read to I/O ports

Function Prototypes:

`outb(port, data)`

- This will take one byte of data and write it to an I/O port

`inb(port, data)`

- This will take one byte of data and read it into an I/O port

`include/mpx/interrupts.h`

This file contains kernel functions related to software and hardware interrupts in the system

Constants include:

- `cli()`
- `sti()`

These constants are used to disable and enable interrupts in the system respectively

Function Prototypes:

`void irq_init(void)`

- This function installs the first 32 interrupt handlers into the IRQ (Interrupt Request Table)

`void pic_init(void)`

- This function initializes the programmable interrupt controllers and performs the necessary remapping of IRQs, it leaves interrupts turned off.

`void idt_init(void)`

- This function creates and installs the interrupt descriptor table

`void idt_install(int vector, void (*handler)(void *))`

- The function installs an interrupt handler

`include/stdlib.h`

This file contains a subset of C standard functions

Function Prototypes:

`int atoi(const char *s)`

- This function converts an ASCII string to an integer literal

`char* itoa(int num, char* str, int base)`

- This function converts a given integer value into an ASCII string

char bcdToChar(unsigned char bcd)

- This function converts a Binary Coded Decimal Value (BCD) into a character

unsigned int intToBCD(unsigned int num)

- This function converts a character into a Binary Coded Decimal Value (BCD)

include/ctype.h

This file contains a subset of C standard function

Function Prototypes:

int isspace(int c)

- This function determines if the given character is a whitespace

int isdigit(char c)

- This function determines if a given character is a digit

include/memory.h

This file contains system-specific dynamic memory functions

Function Prototypes:

void *sys_alloc_mem(size_t size)

- This function dynamically allocates memory in the system

```
int sys_free_mem(void *ptr)
```

- This function frees dynamically allocated memory located within the system

```
void sys_set_heap_functions(void * (*alloc_fn)(size_t), int  
(*free_fn)(void *))
```

- This function installs user-supplied heap-management functions

comhand.c

- This file contains the command handler, which scans for any commands entered into the system

`comhand()`

- `void comhand(void)`
- Authors: Izaak Whetsell
- This function scans for commands to execute in the system which passes them on to be executed in the system
- Parameters:
 - None
- Return Values:
 - None

include/mpx/comexec.h

- This file contains all functions that execute system commands when called in the terminal

Function Prototypes:

`int comexec(char buf[])`

- This function is called to execute any command called in the system, it will scan through the input and determine whether a command is called correctly or incorrectly

`int shutdown(void)`

- This function will terminate the command handling sequence in the system and shut down any processes executing

int help(void)

- This function will print a list of each available command from the current point in the system

int version(void)

- This function prints the compilation date and version of the system

int get_time(void)

- This function retrieves the date from the real time clock (RTC) and displays the current date to the user

int set_time(char buf[])

- This function sets the date of the system using the real time clock (RTC) and displays the results of the command

int get_date(void)

- This function gets the current time of the system using the real time clock (RTC) and displays the time to the user

int set_date(char buf[])

- This function sets the date of the system using the real time clock (RTC) and displays the results of the command to the user

include/mpx/device.h

This header file contains the constants used to identify the device I/O ports in the system

Enums:

device

COM1,

COM2,

COM3,

COM4

Each of these constants contain a memory address which gives the location of the I/O ports to be written to/read from

include/mpx/serial.h

This header file contains the functions and constants used in serial_poll()

Constants include:

- UP_ARROW
- DOWN_ARROW
- LEFT_ARROW
- RIGHT_ARROW
- DELETE_KEY
- BACKSPACE
- NEW_LINE
- CARRIAGE_RETURN
- ESCAPE_SEQUENCE

All of which represent the character sequence of each of their respective keys.

Function Prototypes:

int serial_init(device dev)

- This function initializes the devices connected to the OS for input and output

int serial_out(device dev, const char *buffer, size_t len)

- This function writes the contents of the buffer to the serial port

int serial_poll(device dev, char *buffer, size_t len)

- This function reads the contents of a string to the serial port

void buffer_refresh(char *buffer, int buf_length, int pos)

- This function refreshes the buffer in the terminal window after a key is pressed

include/sys_req.h

This file contains all system request functions and constants

Enums:

op_code

EXIT,

IDLE,

READ,

WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- **INVALID_OPERATION**

- `INVALID_BUFFER`
- `INVALID_COUNT`

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:

```
int sys_req(op_code op, ...)
```

- This function requests a kernel operation within the system to perform an action based on the input parameters.

ctype.c

- ctype.c holds functions relating to the manipulation or extraction of data of the char type.

isdigit()

- int isdigit(char c)
- Author: Chris Jones
- This function determines whether a given character is a digit
- Parameters:
 - char c is the character that is being judged to be or not to be an integer
- Return Values:
 - This function returns an int that represents a boolean.
 - 1 is true and the character is a digit
 - 0 is false and the character is not a digit

include/ctype.h

This file contains a subset of C standard function

Function Prototypes:

int isdigit(char c)

- This function determines if a given character is a numeric digit and returns an integer code of 1 if the character is a digit, and a 0 if not

serial.c

- serial.c holds methods and structures pertaining to the input and output of serial communications.

`buffer_refresh()`

`void buffer_refresh(char *buffer, int buf_length, int pos)`

- Author: Evan Humphrey
- This function keeps track of the buffer in the terminal window as keys are pressed, refreshing every time the buffer is updated.
- Parameters:
 - `char *buffer` is a string of current characters typed by the user
 - `int buf_length` is the length of the passed buffer
 - `int pos` is the position of the user's cursor
- Return Values:
 - None

include/serial.h

This header file contains the functions and constants used in `serial_poll()`

Constants include:

- `UP_ARROW`
- `DOWN_ARROW`
- `LEFT_ARROW`
- `RIGHT_ARROW`
- `DELETE_KEY`

- BACKSPACE
- NEW_LINE
- CARRIAGE_RETURN
- ESCAPE_SEQUENCE

All of which represent the character sequence of each of their respective keys.

Function Prototypes:

`int serial_init(device dev)`

- This function initializes the devices connected to the OS for input and output

`int serial_out(device dev, const char *buffer, size_t len)`

- This function writes the contents of the buffer to the serial port

`int serial_poll(device dev, char *buffer, size_t len)`

- This function reads the contents of a string to the serial port

`void buffer_refresh(char *buffer, int buf_length, int pos)`

- This function refreshes the buffer in the terminal window after a key is pressed

`include/mpx/io.h`

This header file contains kernel macros to write and read to I/O ports

Function Prototypes:

`outb(port, data)`

- This will take one byte of data and write it to an I/O port

`inb(port, data)`

- This will take one byte of data and read it into an I/O port

`include/string.h`

This file contains all functions pertaining to string manipulation and identification

Function Prototypes:

`void* memcpy(void * restrict dst, const void * restrict src, size_t n)`

- This function will copy a desired region of memory to a destination region in memory and will return a pointer to the destination region

`void* memset(void *address, int c, size_t n)`

- This function will take a region of memory and fill it with a desired amount of memory and return a pointer to that memory

`int strcmp(const char *s1, const char *s2)`

- This function compares two strings and returns an integer code specifying the result of the operation

`size_t strlen(const char *s)`

- This function returns the length of a passed-in string

`char* strtok(char * restrict s1, const char * restrict s2)`

- This function will tokenize a string and return

`int contains(const char *str1, const char *str2)`

- This function indicates whether str1 contains str2

`char* substr(char* str1, int index)`

- This function will return the substring of a passed-in string beginning at a starting index

`int isNumeric(char* str)`

- This function determines whether a string is entirely numeric

`int charCount(char* str, char c)`

- This function finds the number of occurrences of a specific character in a given string

`int indexOf(char* str, char searched_char)`

- This function finds the specific index of a desired character in a string

`include/sys_req.h`

This file contains all system request functions and constants

Enums:

op_code

EXIT,

IDLE,

READ,

WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- **INVALID_OPERATION**
- **INVALID_BUFFER**
- **INVALID_COUNT**

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:

int sys_req(op_code op, ...)

- This function requests a kernel operation within the system to perform an action based on the input parameters.

stdlib.c

- Stdlib.c contains all functions that are contained in C standard library functions

itoa()

- char* itoa(int num, char* str, int base)
- Author: Tanner Forbes
- This function converts an integer literal into an ASCII character based on the ASCII table of characters and values
- Parameters:
 - int num is a number to be converted to a character
 - char* str is a where the new character digit will be stored
 - int base is the base of the number system used
- Return Values:
 - char* is the newly converted digit as a character

bcdToChar()

- char bcdToChar(unsigned char bcd)
- Author: Tanner Forbes
- This function converts a Binary Coded Decimal (BCD) value to a character
- Parameters:
 - unsigned char bcd is the Binary Coded Decimal value to be turned into a character
- Return Values:

- Char is the newly converted BCD as a character

`intToBCD()`

- `unsigned int intToBCD(unsigned int num)`
- Author: Chris Jones
- This function converts an integer to a Binary Coded Decimal (BCD) value
- Parameters:
 - `unsigned int num` is the number to convert to BCD.
- Return Values:
 - `Unsigned int` is the BCD value of the passed unsigned integer

`include/stdlib.h`

This file contains a subset of C standard functions

Function Prototypes:

`int atoi(const char *s)`

- This function converts an ASCII string to an integer literal

`char* itoa(int num, char* str, int base)`

- This function converts a given integer value into an ASCII string

`char bcdToChar(unsigned char bcd)`

- This function converts a Binary Coded Decimal Value (BCD) into a character

`unsigned int intToBCD(unsigned int num)`

- This function converts a character into a Binary Coded Decimal Value (BCD)

`include/ctype.h`

This file contains a subset of C standard function

Function Prototypes:

`int isspace(int c)`

- This function determines if the given character is a whitespace

`int isdigit(char c)`

- This function determines if a given character is a digit

string.c

- string.c holds functions relating to the manipulation or extraction of data of strings.

contains()

- int contains(const char *str1, const char *str2)
- Authors: Tanner Forbes and Chris Jones
- Checks to see if a string is inside of another string
- Parameters:
 - const char *s1 is the phrase to check inside of
 - const char *s2 is the string to find
- Return Values:
 - int will be 1 if the second string was located inside the first and 0 if the string was not found or contained in the first.

substr()

- char* substr(char *str,int index)
- Author: Chris Jones
- This function obtains the substring of a string starting at the passed index and proceeding to the end of the string
- Parameters:
 - char *str is the passed string
 - int index is the index to begin the substring
- Return Values:
 - Char * is the substring starting at the passed index and ending with the last character of the string

isNumeric()

- `int isNumeric(char *str)`
- Author: Chris Jones
- Checks to see if the passed string is an assortment of only digit character
- Parameters:
 - `char *str` is the string to check for digits
- Return Values:
 - `int` will be 1 if the passed string is a digit and 0 if any character is not a digit

charCount()

- `int charCount(char str, char c)`
- Author: Chris Jones
- Counts the number of occurrences of the character `c` in a string
- Parameters:
 - `char *str` is the string of characters to count
 - `char c` is the character to count at each occurrence in the string
- Return Values:
 - `int` is the number of occurrences of the character in the string

indexOf()

- `int indexOf(char *str, char searched_char)`

- Author: Tanner Forbes
- Attempts to find the index of a specified character in a string.
- Parameters:
 - char *str is the string to check for the characters position
 - char searched_char is the character to find the position of
- Return Values:
 - int is the index of the found character or -1 for the character not being in the string